National University of Computer & Emerging Sciences

Lecture 16

Threads Introduction

Example: Designing Web server

- Web server accepts client requests for
  - Web pages
  - Images
  - Sound, etc
- It may have thousands of clients accessing it concurrently.

- How would you design it?

Designing Web server

- A web server process
- How you are going to handle client requests?
  - Is it possible with a single process?
  - No.
  - One process per request?
  - No.

Designing Web server

- Process creation
  - Time consuming, resource intensive
- If a new process performs same task then why create a new one?
- Solution:
  - Use threads
  - Reuse process resources

What is a process?

- A computer program in execution on a machine is a process

- More formally:
  - A **Sequential stream of Execution** in its own **address space**

Why use processes?

- Why use processes?
  - Express concurrency
  - Systems have many concurrent jobs going on
  - E.g. Multiple users running multiple shells, I/O, ...

Why use processes?

- A process has an
  - Executable program
  - Stack, Registers
  - Open files
  - Child processes information
  - Accounting information
  - Signal handlers,…

Process Characteristics

- Concept of Process has two aspects.
- A Process is:
  - A Unit of resource ownership:
    - a virtual address space for the process image
    - control of some resources (files, I/O devices...)
  - A Unit of execution - process is an execution path through one or more programs
    - may be interleaved with other processes
    - execution state (Ready, Running, Blocked...) and dispatching priority

Process Characteristics

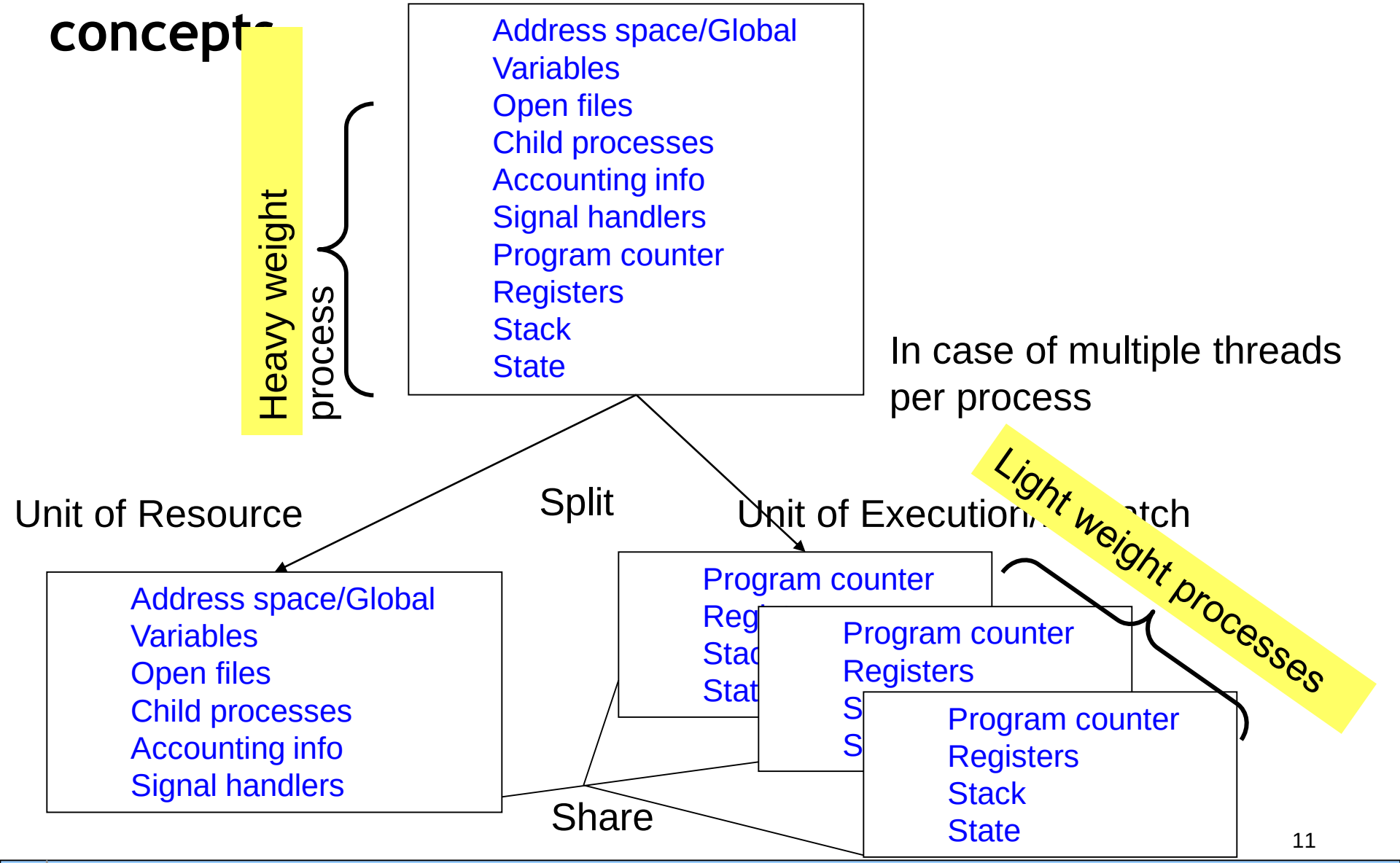
- These two characteristics are treated separately by some recent operating systems:
  - The *unit of resource ownership* is usually referred to as a *process* or *task*
  - The *unit of execution* is usually referred to a *thread* or a “lightweight process”

Threads

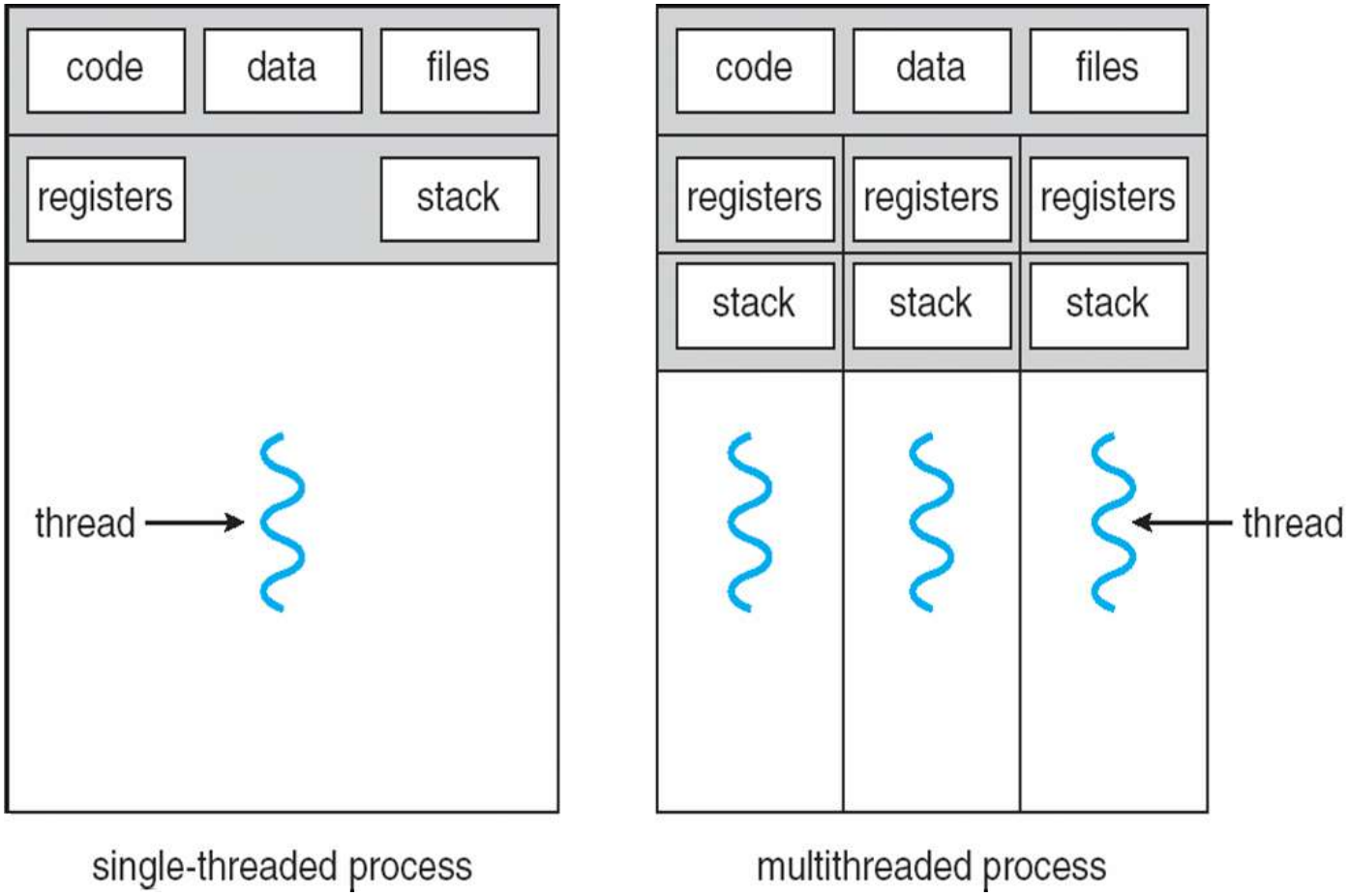
- Processes are used to group resources together
- Threads are the entities scheduled for execution on the CPU
- Threads are also called ***lightweight*** process

- We can have multiple threads in a single process, this is called multithreading.

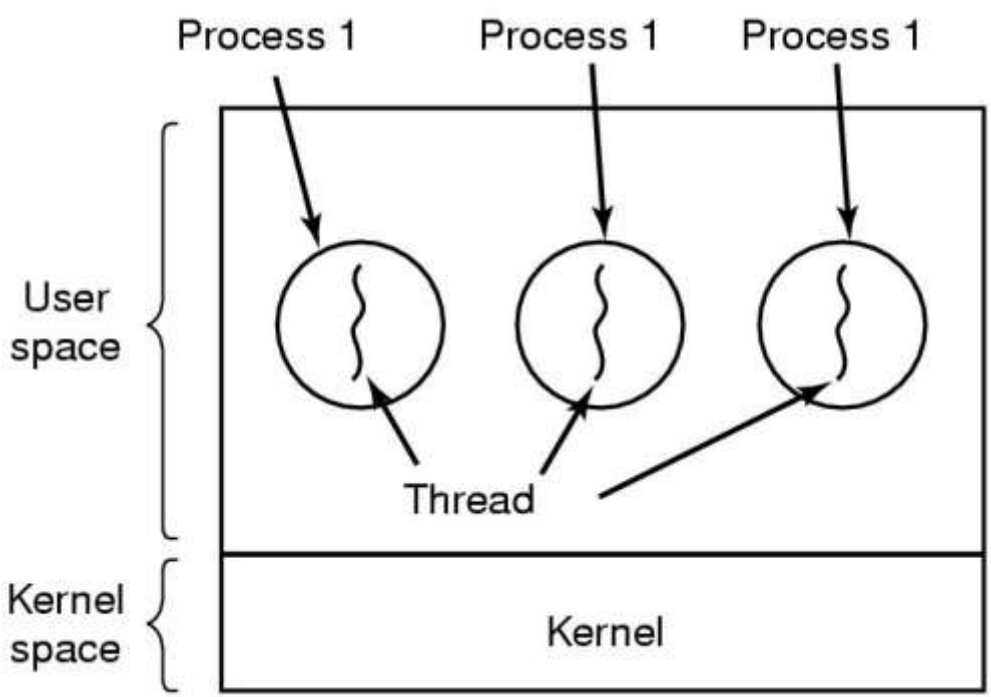
Its better to distinguish between the two concepts



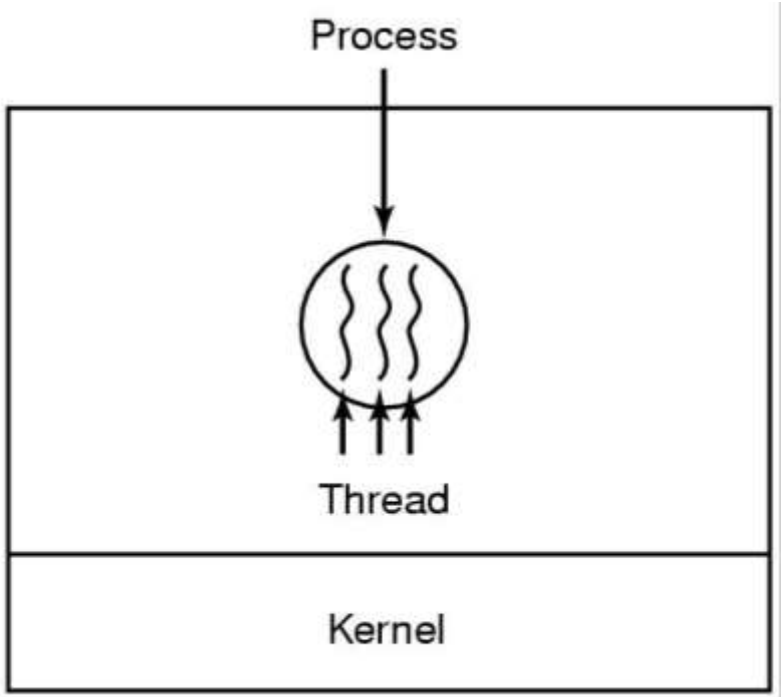
Single and Multithreaded Processes



Process Vs. Threads



(a) Three threads, each running in a separate address space

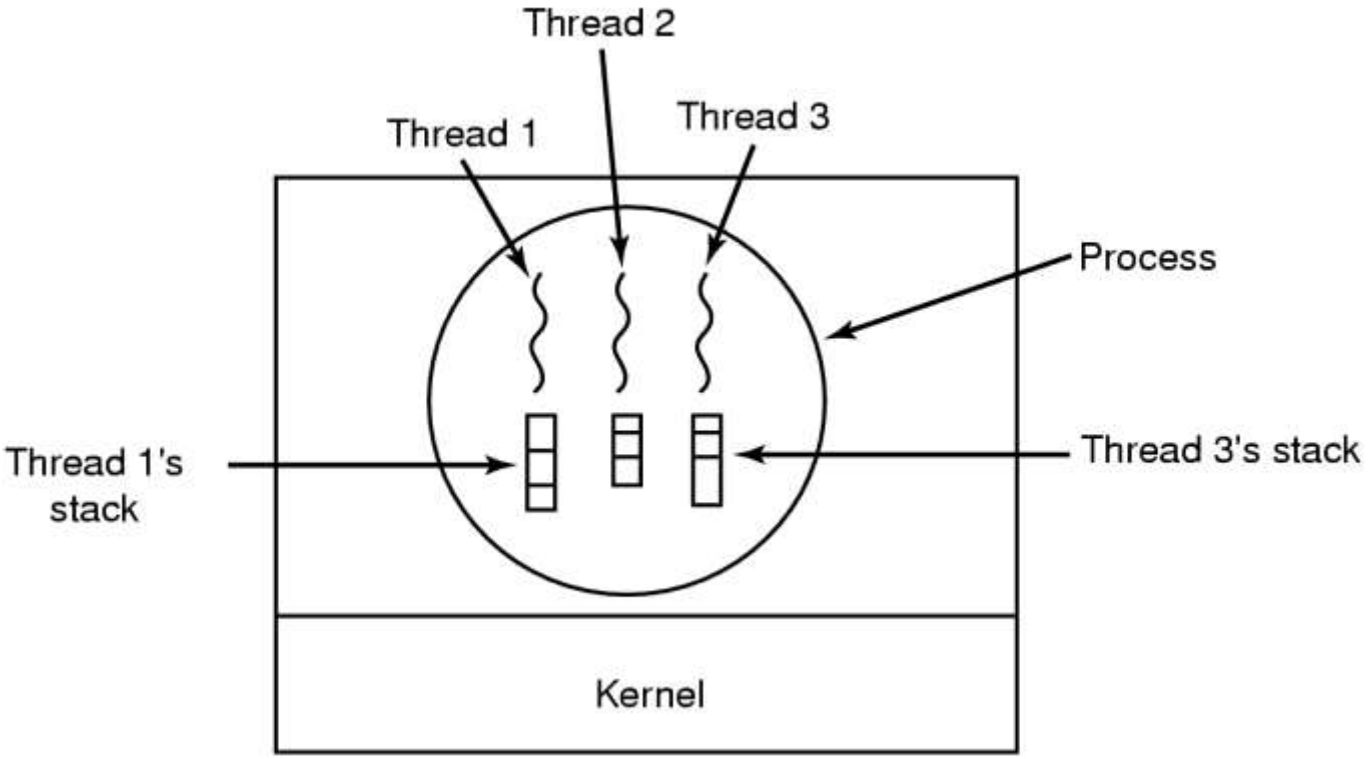


(b) Three threads, sharing the same address space

Threads

- Allow multiple execution paths in the same process environment
- Threads share address space, open files etc
- But have own set of Program counter, Stack etc
- The first thread starts execution with **`int main(int argc, char *argv[])`**
- The threads appear to the Scheduling part of an OS just like any other process

The Thread Model



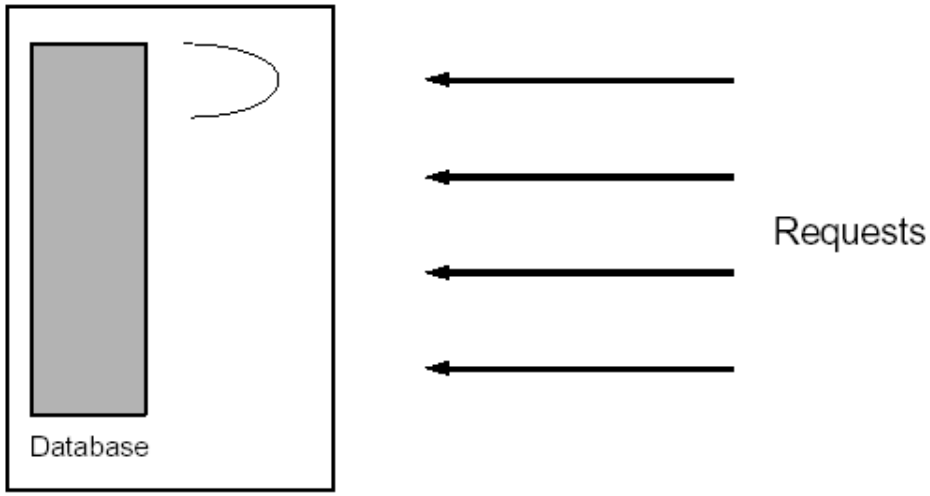
Each thread has its own stack

Thread Usage

- Less time to create a new thread than a process
  - the newly created thread uses the current process address space
  - no resources attached to them
- Less time to terminate a thread than a process.
- Less time to switch between two threads within the same process, because the newly created thread uses the current process address space.
- Less communication overheads
  - threads share everything: address space, in particular. So, data produced by one thread is immediately available to all the other threads
- Performance gain
  - Substantial Computing and Substantial Input/Output
- Useful on systems with multiple processors

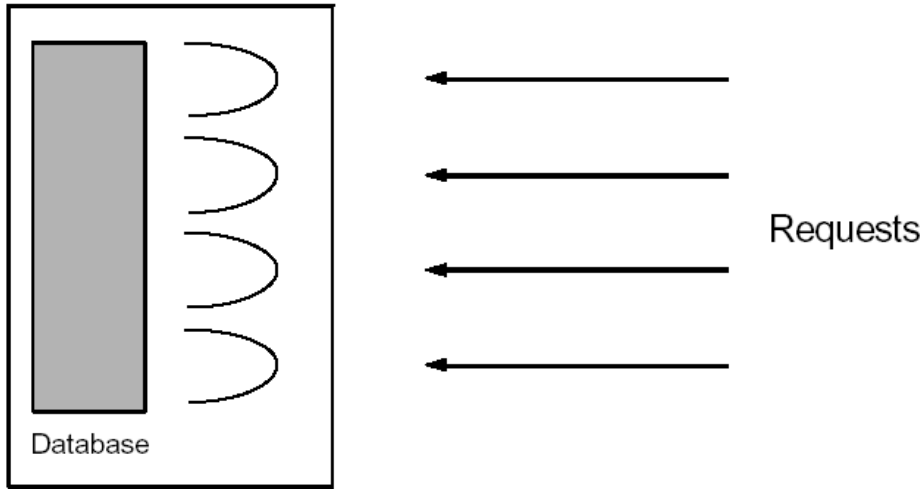
1. Single threaded database server

- Handles multiple clients
  - Either handle the requests sequentially
  - Or multiplex explicitly by creating multiple processes
- Problem with 1
  - Unfair for quick requests, occurring behind lengthy requests
- Problem with 2
  - Complex and error prone
  - Heavy IPC required

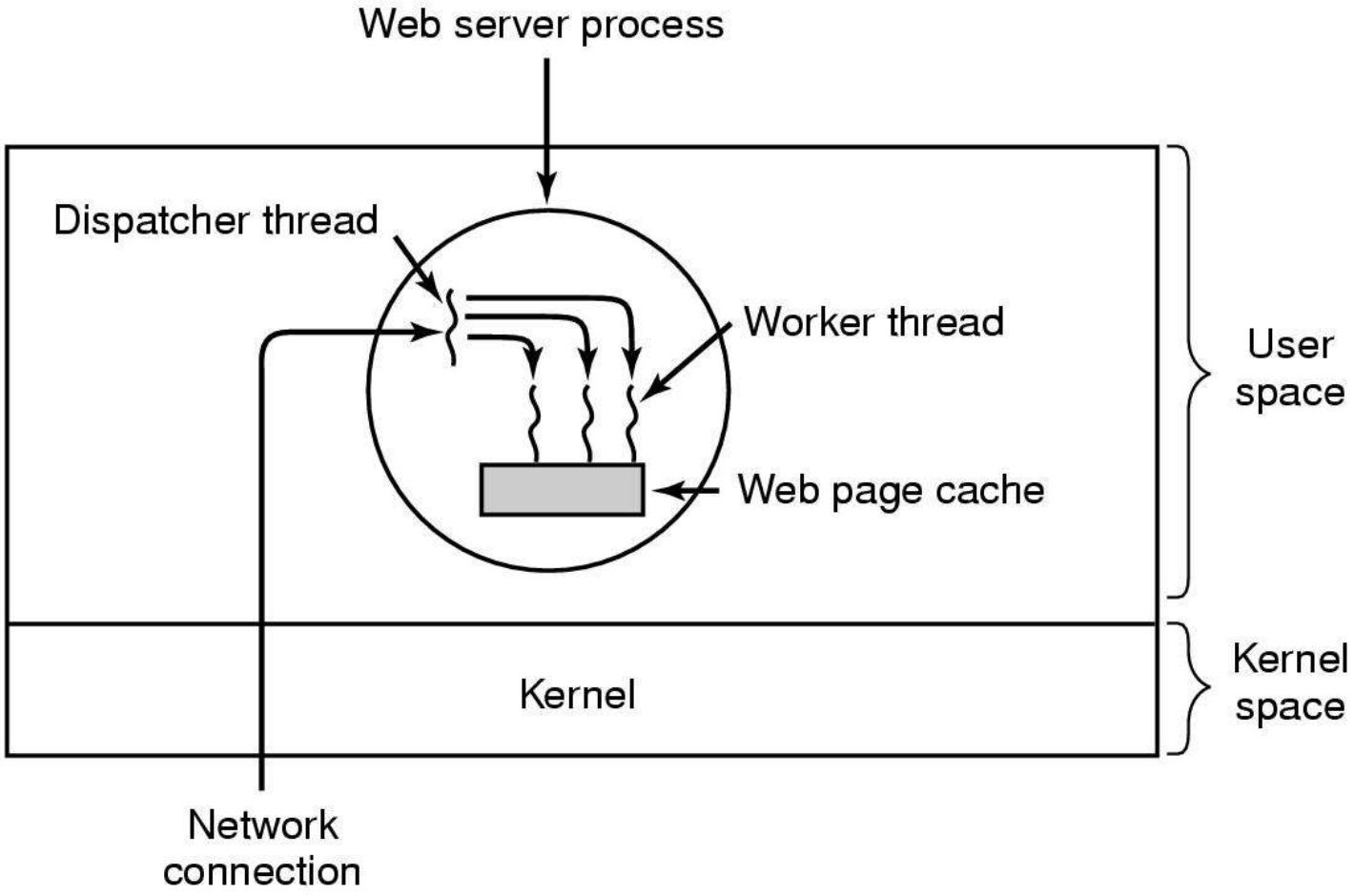


1. Multithreaded database server

- Assign a separate thread to each request
- As fair as in the multiplexed approach.
- The code is as simple as in the sequential approach, since the address space is shared => all variables are available
- Some synchronization of access to the database is required, this is not terribly complicated.



e.g. A multithreaded web server



When to use threads?

- If a program can be organized into discrete, independent tasks which can execute concurrently